SIMATS ENGINEERING
ASSESSMENT TOOLS

**COURSE NAME: Theory of Computation COURSE CODE: CSA1304**

**COURSE OUTCOME COVERED**

**CO5: Design Pushdown Automata and analyze their equivalence with Context-Free Grammars through formal construction and proof techniques. ( BL :3)**

*Assessment Tool Weightage: 40%*

**QUESTIONS: 5 Total Marks:25 Duration : 1 Hour**

# Constraint Based Problem

***Code of Conduct:***

*I Shaik Mohammed Waseem Rayan (192373004) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
*I understand that any violation of academic integrity rules will result in disciplinary*

*action.*

***Signature of the student:***
***Shaik Mohammed Waseem Rayan***

# SIMATS ENGINEERING
# ASSESSMENT TOOLS

**1. Design a Pushdown Automaton that accepts the language $L = \{ a^n b^n \mid n \geq 1 \}$**
**Constraint: You are allowed to use only one stack symbol (besides initial symbol)**

PDA for $L = \{a^n b^n \mid n \geq 1\}$
Constraint: Only one stack symbol besides the initial stack symbol.
Stack Symbols

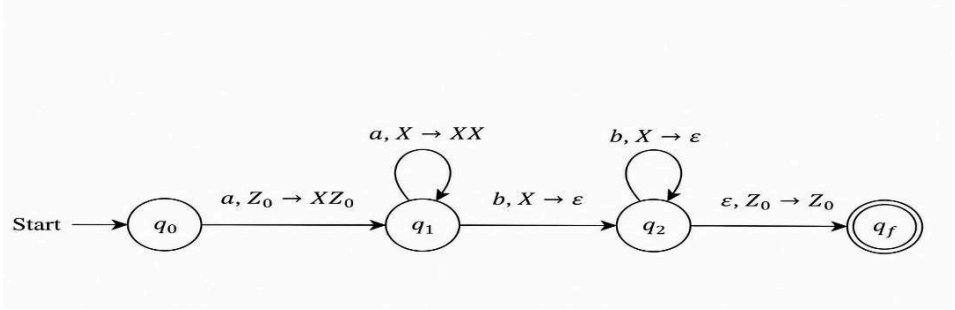
- $Z_0$ = Initial stack symbol
- X = Only additional stack symbol

States

- $q_0$ = Start state
- $q_1$ = Read a's and push X
- $q_2$ = Read b's and pop X
- qf = Final state

Transitions
1. Start reading a:
δ(q0, a, Z0) = (q1, XZ0)
2. For every additional a,
push X: δ(q1, a, X) = (q1, XX)
3. When b starts, pop one X:
δ(q1, b, X) = (q2, ε)
4. For every additional b, pop
one X: δ(q2, b, X) = (q2, ε)
5. When all X's are removed,
accept: δ(q2, ε, Z0) = (qf, Z0)
PDA Transition List
q0 -- a, Z0 → XZ0 --> q1
q1 -- a, X → XX --> q1
q1 -- b, X → ε --> q2
q2 -- b, X → ε --> q2
q2 -- ε, Z0 → Z0 --> qf
Example accepted strings: ab, aabb, aaabbb

Rejected strings: aab, abb, abab, ba




**2. CFG to PDA Conversion with Limited Productions Given a Context-Free Grammar:**

**S → aSb | ab**

**Construct an equivalent PDA.**

**Constraint: PDA must be constructed using only 2 states**

**Given CFG**

S → aSb | ab

**Constraint**

PDA must use only 2 states.

We construct the PDA using:

States = {q0, q1}

Start state = q0

Final state = q1

Initial stack symbol = Z0

**Stack Symbols**

Γ = {Z0, S, a, b}

**Transitions**

**1. Push the start variable S onto the stack:**

δ(q0, ε, Z0) = (q1, SZ0)

**2. Replace S using production S → aSb:**

δ(q1, ε, S) = (q1, aSb)

**3. Replace S using production S → ab:**

δ(q1, ε, S) = (q1, ab)

**4. Match terminal a:**

δ(q1, a, a) = (q1, ε)

**5. Match terminal b:**

δ(q1, b, b) = (q1, ε)

**State Diagram — Copy-Paste Ready**

Start → q0

q0 -- ε, Z0 → SZ0 --> q1

q1 -- ε, S → aSb --> q1
q1 -- ε, S → ab --> q1

q1 -- a, a → ε --> q1
q1 -- b, b → ε --> q1

q1 = Final State

**Example: aabb**

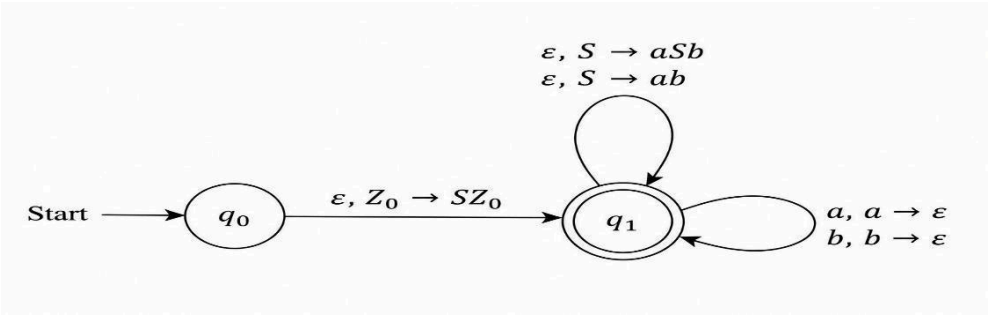
S
↓
aSb
↓
aabb

The PDA expands S using S → aSb, then uses S → ab, and finally matches all terminals.

Therefore:

L(PDA) = L(CFG) = { $a^n b^n$ | n ≥ 1 }

**Number of states = 2: q0 and q1**.




**3. PDA to CFG Conversion with Restricted Variables Convert a given PDA into an equivalent CFG.**

**Constraint: The resulting CFG must use at most 3 variables (non-terminals)**

**Assumption:** We convert the PDA from Question 1, which accepts
$L = \{ a^n b^n \mid n \geq 1 \}$

**Given PDA**

The PDA pushes X for every a and pops X for every b.

q0 -- a, Z0 → XZ0 --> q1
q1 -- a, X → XX --> q1
q1 -- b, X → ε --> q2
q2 -- b, X → ε --> q2
q2 -- ε, Z0 → Z0 --> qf

**Equivalent CFG**

We need **at most 3 variables**.

We can construct the equivalent CFG using just **one variable**:

S → aSb
S → ab

**Derivation**

For ab:
S ⇒ ab
For aabb:
S ⇒ aSb
⇒ aabb
For aaabbb:
S ⇒ aSb
⇒ aaSbb
⇒ aaabbb

**Final CFG**

Variables: {S}

Terminals: {a, b}

Start variable: S

Productions:

S → aSb
S → ab

**Constraint Verification**

Number of variables = 1
Maximum allowed = 3
Therefore, the constraint is satisfied:
1 ≤ 3 ✓
Hence,
CFG = ({S}, {a,b}, P, S)

P = { S → aSb, S → ab }
**Equivalent language:**
L(G) = { $a^n b^n$ | n ≥ 1 }

**4. Turing Machine with Limited Tape Movement**
**Design a Turing Machine to recognize the language**
**L = { ww | w ∈ {0,1}* }**
**Constraint: The tape head can move only to the right (no left movement allowed)**
**Given Language**
L = { ww | w ∈ {0,1}* }
Examples:
ε
00
11
0101
1010
001001
**Constraint**
The tape head can move only to the RIGHT.
No LEFT movement is allowed.
**Result: Impossible**
A standard single-tape Turing Machine whose head can **only move right** cannot recognize
L = { ww | w ∈ {0,1}* }
because once the head moves past a tape cell, it **cannot return to read that symbol again**.
To recognize ww, the machine must:

1. Read the first half w.

2. Determine where the first half ends.
3. Compare the second half with the stored first half.
4. This requires remembering an **arbitrarily long** string.

A finite number of states cannot store an arbitrary-length w.

**Theoretical Reason**

A TM restricted to only rightward movement behaves essentially like a **finite automaton**, because it reads the input only once from left to right and cannot revisit previous symbols.

But:

L = { ww | w ∈ {0,1}* }

is a **non-regular language**.

Therefore:

Right-movement-only TM

↓

Equivalent to finite-state machine

↓

Can recognize only regular languages

↓

{ww} is non-regular

↓

IMPOSSIBLE

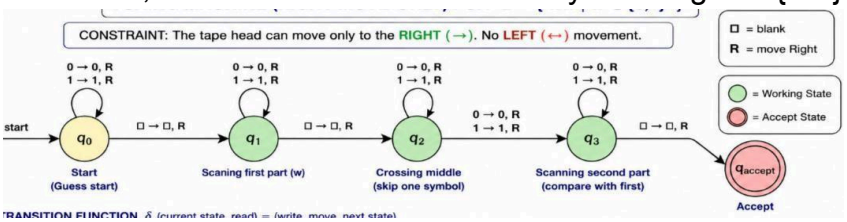
**Final Answer**

No such single-tape Turing Machine exists under the given constraint.

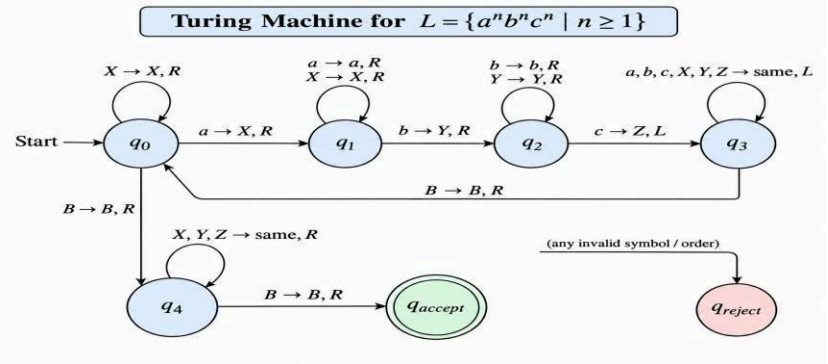
Reason:

The language L = {ww | w ∈ {0,1}*} is non-regular, while a
Turing Machine whose head can move only right behaves like a
finite automaton and can recognize only regular languages.

Therefore, LEFT movement is necessary to recognize {ww}.



**5. Consider a language L = { $a^n b^n c^n$ | n ≥ 1 } Constraint: Design a Turing Machine for L Language**




L = { $a^n b^n c^n$ | n ≥ 1 }

Examples accepted:

abc

aabbcc

aaabbbccc

Examples rejected:

aabcc

abbc

aabbccc

abcabc

**Idea**

The Turing Machine repeatedly:

1. Finds the first unmarked a and changes it to X.
2. Moves right and finds the first unmarked b, changing it to Y.
3. Moves right and finds the first unmarked c, changing it to Z.
4. Returns to the left end.
5. Repeats until all a, b, and c are marked.
6. If only X, Y, and Z remain, **accept**.

**Tape Symbols**

Input symbols: {a, b, c}

Tape symbols: {a, b, c, X, Y, Z, B}

Where:

X = marked a

Y = marked b
Z = marked c
B = blank

**States**

q0 = Start / find an unmarked a
q1 = Find corresponding b
q2 = Find corresponding c
q3 = Return to the left
q4 = Check whether all symbols are marked
qaccept = Accept
qreject = Reject

**Transition Logic**

q0:
a → X, R, q1
X → X, R, q0
Y/Z → reject
q1:
a → a, R, q1
X → X, R, q1
b → Y, R, q2
c → reject
q2:
b → b, R, q2
Y → Y, R, q2
c → Z, L, q3
a → reject
q3:
a → a, L, q3
b → b, L, q3
c → c, L, q3
X → X, L, q3
Y → Y, L, q3
Z → Z, L, q3
B → B, R, q0
When no unmarked a remains:

q0:
B → B, R, q4
**Final Check**
q4:
X → X, R, q4
Y → Y, R, q4
Z → Z, R, q4
B → B, R, qaccept
a/b/c → qreject
**State Diagram — Copy-Paste Ready**
Start → q0

q0 -- a → X, R --> q1
q0 -- X → X, R --> q0

q1 -- a → a, R --> q1
q1 -- X → X, R --> q1
q1 -- b → Y, R --> q2

q2 -- b → b, R --> q2
q2 -- Y → Y, R --> q2
q2 -- c → Z, L --> q3

q3 -- a,b,c,X,Y,Z → same, L --> q3
q3 -- B → B, R --> q0

q0 -- B → B, R --> q4

q4 -- X,Y,Z → same, R --> q4
q4 -- B → B, R --> qaccept

Invalid symbol/order → qreject
**Final Answer**
The key technique is **marking one a, one b, and one c in every cycle** using X, Y, and Z.

SIMATS ENGINEERING ASSESSMENT TOOLS

**RUBRICS FOR CALCULATION**

| Criteria | Weightage | Level 4 – Exemplary (5 Marks) | Level 3 – Proficient (4 Marks) | Level 2 – Developing (2–3 Marks) | Level 1 – Beginning (0– 1 Mark) |
|---|---|---|---|---|---|
| **Conceptual Understanding** | **30** | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps | Basic understanding with some misconceptions | Poor or incorrect understanding |
| **Accuracy of Answer** | **25** | Completely correct answer with no errors | Mostly correct with minor errors | Partially correct with noticeable errors | Incorrect or irrelevant answer |
| **Application of Knowledge** | **20** | Correctly applies concepts to solve the question/problem | Applies concepts with minor mistakes | Limited or partially correct application | No or incorrect application |
| **Completeness of Response** | **15** | Covers all required parts of the question | Covers most parts with minor omissions | Covers only some parts | Incomplete or missing response |
| **Clarity & Presentation** | **10** | Clear, concise, and well-organized answer | Understandable with minor issues | Somewhat unclear or poorly structured | Difficult to understand |